

Reporte Detallado de Cobertura y Suite de Pruebas — DIZI

Este documento detalla exhaustivamente cada una de las pruebas automatizadas (unitarias, integración y de sistema E2E) implementadas en el proyecto **Dizi**, describiendo las funciones que analizan, los escenarios que simulan y las aserciones que realizan.

Resumen Ejecutivo de Pruebas

* **Framework de Pruebas:** Vitest v4.1.10 (nativo de Vite) y Playwright (para E2E). * **Total de Archivos de Pruebas (Suites):** 6 suites ejecutadas en Vitest + 1 suite E2E en Playwright. * **Total de Casos de Prueba Unitarios e Integración:** 66 casos (100% exitosos). * **Total de Escenarios de Sistema E2E:** 5 escenarios (Playwright). * **Tiempo de Ejecución total de la suite de consola:** 576 milisegundos.

1. Detalle por Suite de Pruebas (Archivos de Código)

A continuación, se detalla qué evalúa cada uno de los archivos de prueba creados en la carpeta ``src/lib/__tests__`` y ``tests``:

Suite A: ``types.test.ts`` (41 casos de prueba)

Este archivo contiene la lógica de negocio central sobre las suscripciones del comerciante, las degradaciones de planes y los formatos de plantillas. * **Archivo de código probado:** `src/lib/types.ts` * **Casos que verifica:** * ``daysSinceExpiry`` (4 casos): * Retorna ``null`` si la tienda no tiene fecha de vencimiento configurada. * Retorna ``0`` si el vencimiento coincide exactamente con el día de hoy. * Retorna la cantidad exacta de días transcurridos desde que venció (ej: 3 días de vencido). * Retorna un número negativo si la suscripción aún está vigente (ej: -3 días para expirar). * ``daysUntilExpiry`` (3 casos): * Retorna ``null`` si no hay fecha de expiración. * Retorna la cantidad exacta de días restantes hasta el vencimiento (número positivo). * Retorna número negativo si la suscripción ya caducó. * ``getEffectivePlan`` (5 casos): * Mantiene el plan ``"semilla"`` si era el plan base. * Mantiene el plan original si no tiene expiración. * **CP-06:** Conserva el plan original de pago si expira hoy (está dentro del periodo de gracia de 3 días). * **CP-07:** Conserva el plan original si venció hace 3 días (último día de gracia). * **CP-08:** Cambia el plan efectivo a ``"semilla"`` si venció hace 4 días (gracia expirada). * ``getEffectiveProductLimit`` (3 casos): * Retorna el límite semilla (20 productos) para plan semilla. * Retorna el límite original (ej. 200 productos para Pro) si el

plan está vigente. * Aplica el límite semilla (20) si el plan ya expiró y superó el periodo de gracia. * **`isSubscriptionExpired` (4 casos):** * Retorna `false` para tiendas en plan semilla. * Retorna `false` si no tiene fecha de expiración. * Retorna `false` si la expiración es a futuro. * Retorna `true` si la expiración es en el pasado. * **`modelGraceDaysLeft` (4 casos):** * Retorna `null` si es semilla. * Retorna `null` si el plan de pago está vigente. * Retorna los días restantes de gracia para mantener la plantilla visual (máximo 15 días tras vencer). * Retorna `0` si el vencimiento superó los 15 días. * **`shouldUseSemillaModel` (2 casos):** * Retorna `false` si está dentro de la gracia de 15 días del modelo. * Retorna `true` si ya pasaron los 15 días y se debe forzar el modelo semilla. * **`getEffectiveModel` (2 casos):** * Retorna `"minimalista"` (modelo semilla) si la gracia del modelo ya expiró. * Retorna el modelo premium configurado (ej: `"boutique"`) si aún está dentro de los 15 días de gracia. * **`isPlanActive` (4 casos):** * Retorna `true` para semilla. * Retorna `false` si no tiene fecha de expiración y no es semilla. * Retorna `true` si la expiración es futura. * Retorna `false` si la expiración es pasada. * **`getBioLinksLimit` (2 casos):** * Devuelve `3` enlaces rápidos como límite si la tienda está bajo plan semilla. * Devuelve `Infinity` si está en un plan premium activo. * **`canUsePremiumBioFeatures` (2 casos):** * Devuelve `false` para plan semilla. * Devuelve `true` para planes premium. * **`getImageSpec` (3 casos):** * Mapea el modelo `"minimalista"` al layout `"grid"` (escala cuadrada 1:1). * Mapea el modelo `"vibrante"` al layout `"overlay"` (escala vertical 3:4). * Mapea el modelo `"portada"` al layout `"banner\_grid"` (escala panorámica 16:7). * **`checkImageRatio` (3 casos):** * Retorna `status: "ok"` si las dimensiones de la imagen coinciden con la relación recomendada. * Retorna `status: "warning"` avisando de recorte en los lados si es demasiado ancha. * Retorna `status: "warning"` avisando de recorte arriba/abajo si es demasiado alta.

▣ Suite B: `whatsapp.test.ts` (5 casos de prueba)

Verifica el formateo de precios en la moneda nacional (Soles) y la correcta codificación del mensaje de WhatsApp del checkout. * **Archivo de código probado: src/lib/whatsapp.ts** * **Casos que verifica:** * **`formatPrice` (4 casos):** * **CP-01:** Formatea montos positivos de manera correcta (ej. `25.5` -> `"S/ 25.50"`). * **CP-02:** Retorna `"A consultar"` si el precio es `0`. * **CP-03:** Retorna `"A consultar"` si el precio es `null` o `undefined`. * **CP-04:** Redondea centavos adecuadamente con dos decimales (ej. `0.005` -> `"S/ 0.01"`). * **`buildWaUrl` (1 caso):** * **CP-05:** Limpia caracteres no numéricos del teléfono (remueve espacios y guiones) y codifica los caracteres especiales del mensaje para la API de WhatsApp (ej. `¿` y `?` a entidades URL).

▣ Suite C: `image-utils.test.ts` (6 casos de prueba)

Comprueba el comportamiento del procesador de imágenes en el cliente (que reduce la carga en el servidor convirtiendo a WebP antes de subir). * **Archivo de código probado: src/lib/image-utils.ts** * **Casos que verifica:** * **`convertImageToWebP` (4 casos):** * Convierte un archivo de imagen válido al formato base64 WebP respetando el tamaño si no excede los límites. * Redimensiona proporcionalmente el ancho a un máximo de 2048px si la imagen original es más ancha (ej: 3000px -> 2048px). * Redimensiona

proporcionalmente el alto a un máximo de 2048px si la imagen original es más alta. * Lanza un error controlado si el archivo de imagen está dañado o no se puede procesar. * ``convertImageUrlToWebP` (2 casos)`: * Devuelve inmediatamente la URL si ya es una data URL (base64) evitando reprocesamientos. * Descarga una imagen remota vía CORS anónimo y la transforma en data URL WebP.

▮ Suite D: ``error-capture.test.ts`` (4 casos de prueba)

Monitorea la captura de excepciones globales y promesas rechazadas para el diagnóstico del sistema. * **Archivo de código probado:** `src/lib/error-capture.ts` * **Casos que verifica:** * Retorna ``undefined`` si no se ha capturado ningún error. * Captura y consume un error global (``errorListener``) limpiándolo del buffer tras leerlo para evitar fugas de memoria. * Captura y consume un rechazo de promesa no controlado (``unhandledrejection``). * Descarta el error automáticamente si se intenta consumir después de 5 segundos de haber ocurrido (TTL - Time to Live).

▮ Suite E: ``utils.test.ts`` (3 casos de prueba)

Prueba la función utilitaria ``cn`` para la mezcla condicional de clases CSS. * **Archivo de código probado:** `src/lib/utils.ts` * **Casos que verifica:** * Combina nombres de clases CSS normales. * Ignora elementos vacíos, nulos, indefinidos o booleanos falsos. * Resuelve conflictos de clases de Tailwind CSS usando ``twMerge`` (ej: si pasas ``p-4 p-8`` conserva la de mayor precedencia ``p-8``).

▮ Suite F: ``error-page.test.ts`` (2 casos de prueba)

Verifica el renderizador estático de HTML para caídas del servidor. * **Archivo de código probado:** `src/lib/error-page.ts` * **Casos que verifica:** * Retorna un documento HTML bien formado (``<!doctype html>`,`<html>`,`<body>``). * Muestra el mensaje informativo en inglés y el código Javascript para recargar la página.

▮ Suite G: ``security.test.ts`` (5 casos de prueba de Integración)

Valida la capa de seguridad y RPCs del backend (Supabase) simulando la respuesta del servidor. * **Archivo de código probado:** `tests/integration/security.test.ts` * **Casos que verifica:** * **CP-12 (RLS):** Garantiza que un tenant no pueda consultar las filas de productos que pertenecen a otra tienda (aislamiento de datos). * **CP-11 (check_invite):** Valida que la llamada al RPC con un token usado retorne un array vacío. * **CP-10 (check_invite):** Valida que al enviar un token activo y no usado, el RPC devuelva la configuración del plan. * **CP-13 (Trigger de Privilegios):** Emula que si un atacante intenta inyectar el rol ``super_admin`` durante el

registro, el disparador lo degrade inmediatamente a ``store_owner``. * **CP-14 (Tienda despublicada):** Valida que la consulta a una tienda que no está marcada como activa retorne un error controlado de "tienda no publicada".

2. Pruebas de Sistema / E2E (Playwright)

Estas pruebas simulan la experiencia real del usuario en el navegador web de manera automatizada: * **Archivo de especificaciones:** `tests/e2e/catalogo.spec.ts` * **Escenarios definidos:** * **`E2E-01` (Publicación de Catálogo):** Loguea al dueño, activa la casilla de publicación, guarda y comprueba que al navegar a la ruta pública ``/t:slug`` se rendericen las tarjetas de producto y los precios en soles. * **`E2E-02` (Registro de invitación):** Simula el flujo del formulario de registro ingresando un código de invitación válido (éxito) y uno vencido (bloqueo por alerta). * **`E2E-03` (Pedido por WhatsApp):** Entra al catálogo, agrega dos productos al carrito, abre el checkout, ingresa datos y simula el clic de envío a WhatsApp, verificando que la URL destino contenga la estructura codificada con el celular del comercio. * **`E2E-04` (Libro de Reclamaciones):** Simula a un consumidor abriendo el libro en el catálogo (1 solo clic), llenando sus datos personales (wizard paso 1) y el reclamo (paso 2), enviando, y luego al administrador iniciando sesión para responder y resolver el reclamo desde su panel. * **`E2E-05` (Restricciones del Plan):** Simula el inicio de sesión de un comerciante en plan semilla (gratuito) y comprueba que las pestañas e inputs de diseño premium aparezcan bloqueados (disabled) con una llamada a mejorar de plan.

3. Trazabilidad de Casos de Prueba (Matriz)

Esta tabla resume la trazabilidad de los requisitos analizados en las pruebas unitarias y de integración:

Código de Caso	Módulo Evaluado	Tipo de Prueba	Requisito Relacionado	Resultado Esperado
:---	:---	:---	:---	:---
CP-01	<code>`formatPrice`</code>	Unitaria	RF-03 (Formateo precio)	Agrega prefijo <code>`S/ `</code> y dos decimales (<code>`S/ 25.50`</code>).
CP-02	<code>`formatPrice`</code>	Unitaria	RF-03 (Formateo precio)	Si el precio es <code>`0`</code> muestra <code>`"A consultar"`</code> .
CP-03	<code>`formatPrice`</code>	Unitaria	RF-03 (Formateo precio)	Si el precio es <code>`null`/`undefined`</code> muestra <code>`"A consultar"`</code> .
CP-04	<code>`formatPrice`</code>	Unitaria	RF-03 (Formateo precio)	Redondea decimales al valor más cercano (<code>`S/ 0.01`</code>).

CP-05	`buildWaUrl`	Unitaria	RF-02 (Mensaje WhatsApp)	Genera enlace `wa.me` limpio con el texto codificado.
CP-06	`getEffectivePlan`	Unitaria	RF-06 (Periodo de gracia)	El día de vencimiento se conserva el plan de pago.
CP-07	`getEffectivePlan`	Unitaria	RF-06 (Periodo de gracia)	El día 3 de vencido aún se mantiene el plan de pago.
CP-08	`getEffectivePlan`	Unitaria	RF-06 (Periodo de gracia)	El día 4 de vencido degrada efectivamente a `"semilla"`.
CP-09	`getEffectiveProductLimit`	Unitaria	RF-05 (Límites de plan)	Aplica límite correspondiente al plan efectivo (20 o 200).
CP-10	`check_invite`	Integración	RF-04 (Registro por token)	Retorna datos del plan para tokens válidos.
CP-11	`check_invite`	Integración	RF-04 (Registro por token)	Retorna array vacío para tokens ya usados o vencidos.
CP-12	`stores` / `products`	Integración	RF-07 (Aislamiento RLS)	Consulta de Tienda A no puede leer filas de Tienda B.
CP-13	Trigger sync	Integración	RF-08 (Mitigación roles)	Degrada rol `super_admin` inyectado a `store_owner`.
CP-14	`get_public_store`	Integración	RF-01 (Publicar catálogo)	Niega acceso a catálogos despublicados.